

HAIRSPRING: Authority outside the model

A replayable agent harness for work that must survive its own confidence

Tensorbend Instinct

September 2026

Abstract

Language models can use tools, revise artifacts, and run for long periods. The unresolved design question is who decides what counts. If the same model proposes the work and declares it correct, a fluent failure can cross the system boundary as success. HAIRSPRING moves that authority into a replayable substrate. Executable checks close missions. A critic in a fresh context attempts to refute the submitted artifact. A single writer records canonical, hash-linked events. World and memory services admit validated state. A pinned scorer promotes policy changes only after held-out trials. On a matched ten-task SWE-bench-Live subset, HAIRSPRING resolved 9 tasks and SWE-agent resolved 3 with the same model family and host-side grading. HAIRSPRING used fewer agent steps, more wall time, and more provider-reported cost. This is a bounded harness result, not a general leaderboard claim. It supports a narrower point: the common think-act-observe loop is not the differentiator. Authority is.

1 Introduction

Agent harnesses commonly wrap a language model in the same control loop: provide a goal, expose tools, append observations, and continue until the model submits an answer. The loop is useful. It is also incomplete. It says how work proceeds, but not who may declare the work correct, which effects become shared state, how a crashed mission resumes, or what evidence can replace the current policy.

These decisions matter because model confidence is not an execution boundary. A model can submit a plausible answer after running the wrong test, repair a symptom while preserving the defect, or repeat a failed action because the environment never proved that success was possible. A longer context does not fix this class of error. It gives the same component more history while leaving its authority unchanged.

HAIRSPRING separates work from judgment. The model proposes actions and artifacts. Mechanisms outside the working context decide whether an effect enters durable state, whether the mission closes, and whether a policy change replaces the incumbent. The event record connects those mechanisms. It is operational state rather than a post-hoc transcript: recovery, inspection, memory extraction, accounting, and outer-loop scoring read the same history.

Contributions. This paper makes four contributions.

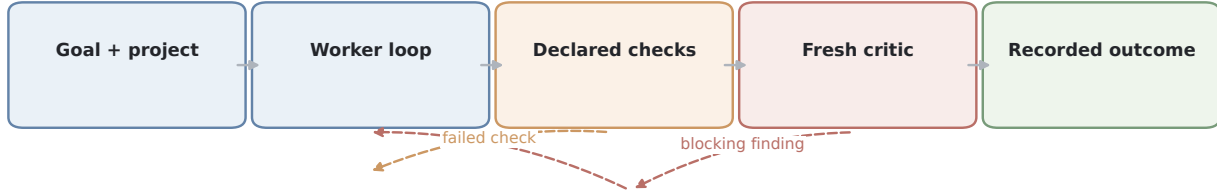
1. It defines an authority architecture for long-running agents: checker, critic, event record, world service, memory service, and scorer have distinct powers.
2. It implements the architecture as a 14-crate Rust workspace with an installed terminal path, typed plugin protocol, process confinement, replay, bounded child agents, and policy-level self-improvement.
3. It reports a matched ten-task comparison against SWE-agent, including steps, wall time, cost, failures, and the limits of the comparison.
4. It turns failures at subsystem seams into an evaluation method that distinguishes local mechanism tests, installed-path tests, live artifact trials, and matched external evaluation.

2 Design problem: the model cannot be its own authority

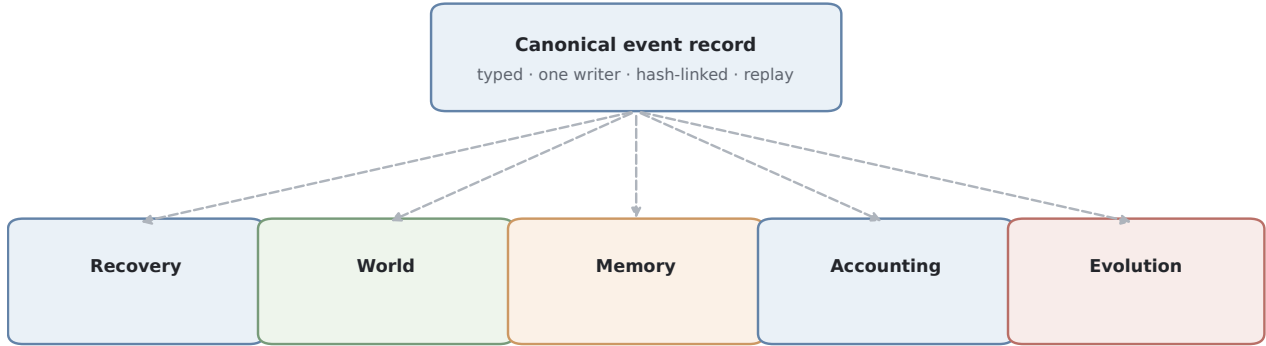
A useful harness must answer five questions that the inner loop cannot settle by assertion.

Did the requested artifact work? The answer belongs to executable checks that run against the judged workspace, followed by an independent attempt to find a blocking defect.

(a) Mission workflow



(b) The same record feeds five authority planes



(c) Authority boundaries



Figure 1: The HAIRSPRING architecture. Panel (a) follows a mission from goal to recorded outcome. Failed checks and grounded critic findings return to the worker. Panel (b) shows how one event record feeds recovery, shared state, memory, accounting, and evolution. Panel (c) names the authority boundaries. The model proposes; separate mechanisms close, admit, and promote.

What happened before a crash? The answer belongs to a durable event stream with explicit ordering, payload identity, and recovery semantics. A reconstructed prompt is not enough because it can omit tool state, counters, costs, or partial effects.

Which effects become shared truth? The answer belongs to a validation boundary. Child agents may propose updates, but proposals do not become world state merely because a model wrote them.

Which memories deserve reuse? The answer belongs to records with provenance and observable reuse signals. Retrieval relevance is different from correctness; the system preserves that distinction.

Which change improves the harness? The answer belongs to repeated trials under a pinned scorer. A candidate cannot choose the test, grade the result, and promote itself.

The architecture follows from these answers. The model remains free to search, edit, call tools, delegate, and revise. It cannot write a passing verdict, append directly to the canonical log, bypass world validation, or publish a promotion record without the scorer path.

3 System overview

A mission binds a goal to a project root, provider configuration, tool registry, checks, budgets, and a durable run directory. The worker calls typed tools through a newline-delimited JSON protocol. The loop records requests and results as events.

Large payloads are stored by content hash. The worker submits an artifact together with the checks that define success. The checker restores the judged workspace, runs those checks, and invokes the critic. A failure returns evidence for repair. A check pass with no blocking critic finding records a verified outcome.

Five planes surround this lifecycle.

Log plane. Canonical events, one writer, segmented persistence, torn-tail recovery, and replay.

World plane. Validated proposals, shared artifacts, and observable reuse across agents.

Memory plane. Extracted records, provenance, retrieval, and citation feedback.

Accounting plane. Model calls, tool work, child-agent work, limits, and billed cost.

Evolution plane. Candidate policies, pinned assays, canaries, promotion, and rollback.

This organization is model-agnostic. Providers are configuration. The authority boundaries do not depend on a particular model family, and a model change is logged as an infrastructure change rather than silently credited to policy fitness.

4 Mission execution

4.1 Typed tools and a generated surface

Tools run as processes behind a typed protocol. The kernel registers capabilities and generates the mission surface from that registry. This avoids maintaining one tool list in the prompt and another in execution code. Requests and results retain the registered tool name, arguments, timing, outcome, and payload references in the event stream.

The edit path uses anchored or patch-based mutations rather than unconstrained file replacement. Terminal execution occurs inside a platform confinement layer. On Linux, the runner uses bubblewrap; on macOS, it emits a Seatbelt profile. Startup fails when the required confinement mechanism is unavailable. The project and required caches remain writable while system paths remain read-only.

4.2 Submission is a transaction

A submission carries the answer and its declared checks together. The checker judges a restored workspace, not whichever transient directory happens to be current. This ordering closes a race found in an earlier path where answer text could become visible before the final check declaration was persisted. The regression test reproduces the original multi-step failure rather than testing only the repaired function.

4.3 The critic can block but cannot approve

The critic receives the goal and submitted artifact in a fresh context without the worker's rationale. Its job is to identify a grounded blocking defect. In the production path it runs under a read-only operating-system identity. An abnormal critic exit fails closed. A critic report can return work for repair; it cannot by itself turn failed executable checks into a pass.

Verdict records distinguish verified work from capped retries or verifier malfunction. This distinction prevents an operational limit from being rendered as successful judgment.

5 The event record

A transcript records conversation. A run record must reconstruct action. HAIRSPRING uses typed, canonical events, content-addressed payloads, segmented streams, and a single append authority. Recovery reads a consistent prefix, detects torn tails, and restores mission state. Plugins return effects to the loop rather than appending events directly.

The constraint changes the semantics of resume. A resumed mission carries the previous transcript, tool results, counters, ledger, and billed cost. It does not start a cosmetically similar mission under a new identity. Mid-run steering, provider changes, child-agent work, and scoring evidence remain events in the same history.

Evidence. Storage tests inject torn tails, duplicate clock values, corrupt registry markers, dead plugins, interrupted migrations, and replay after promotion. The relevant proofs live in the crate test suites at the evaluated source revision.

A mission is reconstructed from events, not from model memory

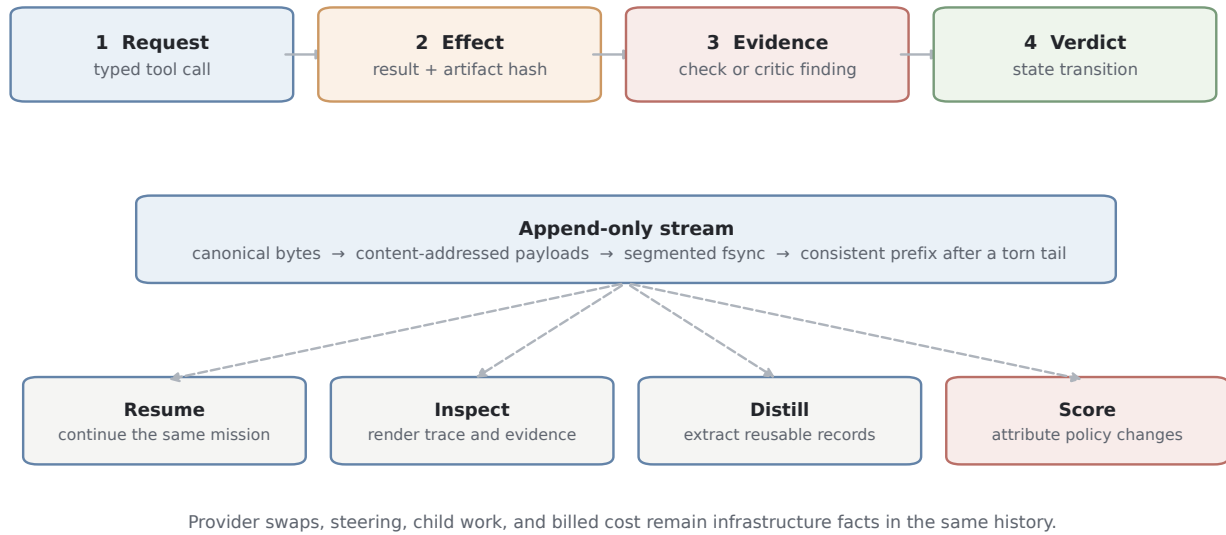


Figure 2: Event anatomy and downstream uses. Requests, effects, evidence, and verdicts enter one durable stream. Resume, trace rendering, memory extraction, and scoring consume the same record.

6 Shared work without shared authority

6.1 Bounded child agents

A parent can start asynchronous child missions with explicit step limits. Each child writes to its own stream under the parent’s log root, books its work to the parent, and joins before mission close. Child agents cannot recursively delegate. The depth boundary is enforced by the spawned tool surface, not by a natural-language request.

6.2 World proposals

Agents submit proposed effects to the world service. Validation separates a model’s report from shared state. Accepted artifacts can be discovered by another child without a direct message between them. Reuse remains observable in the world record.

6.3 Memory with provenance

The memory service stores extracted records in a global knowledge base. A served record that a later mission cites receives positive feedback; a served record that the mission ignores receives negative feedback. This is a reuse signal, not a truth score. The original source and mission identity remain attached to the record.

The system has end-to-end tests for writes, recall, and cross-child reuse. A broader behavioral question remains open: whether a capable live model selects the right record as the knowledge base grows.

7 Policy-level self-improvement

The inner loop repairs a task artifact. The outer loop changes harness policy. A proposer receives durable prior policy, traces, scores, reflections, and frontier state. It emits a candidate. Candidate and parent run on held-out tasks under a scorer pin established before mutation. Canaries detect scorer drift. Promotion records retain candidate and parent hashes, assay conditions, traces, scores, decision, and reason.

Two observed promotions preserved equal pass counts while reducing total steps from 94 to 52 and then from 76 to 41. These were small, two-task assays. They show that the promotion path can produce and install a winner. They do not estimate

Table 1: Observed policy promotions. Each row compares a candidate with its parent on two held-out tasks.

Assay	Candidate steps	Parent steps	Reduction
Cycle 1	52	94	44.7%
Cycle 2	41	76	46.1%

Evidence strengthens as it moves outward from code to outcome

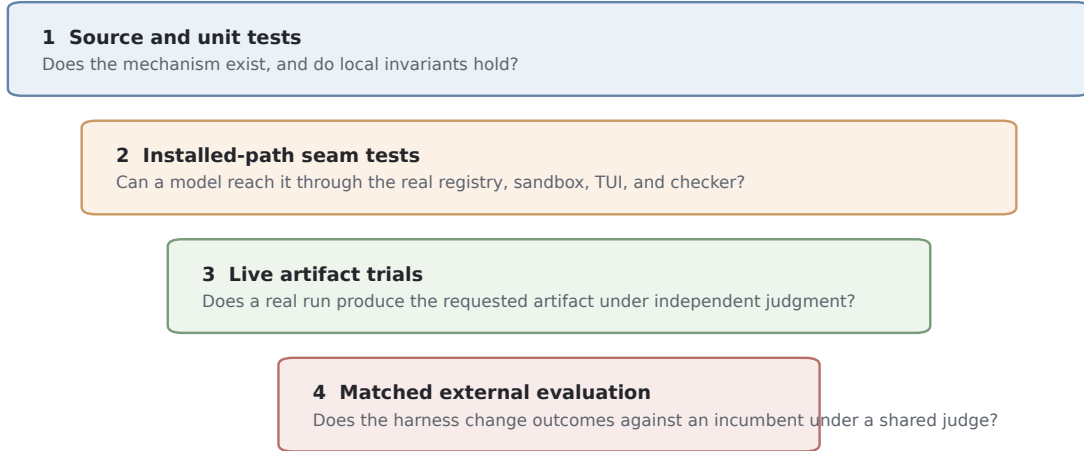


Figure 3: The evidence ladder used in this paper. A large unit suite does not substitute for an installed-path result, and a live result does not by itself establish a general benchmark claim.

a population-level improvement rate. The current system changes prompts and policy artifacts; it does not yet author and safely promote changes to its own Rust implementation.

8 Evaluation method

The evaluation separates four kinds of evidence. Source and unit tests establish that a mechanism exists and preserves local invariants. Installed-path seam tests establish that the terminal, registry, sandbox, checker, and rendered state connect the mechanism to a real mission. Live artifact trials establish that a real model can produce and pass an independently judged artifact. Matched external evaluations compare harness outcomes under a shared judge.

The principal comparison uses ten SWE-bench-Live tasks with the same model family and host-side fail-to-pass checks. Each system operated through its native interface. Raw records identify task, submission status, outcome, steps, wall time, and cost. The comparison is a curated subset, not a leaderboard sample. There are no repeated seeds or confidence intervals.

9 Matched software-engineering result

HAIRSPRING’s failure was a two-hour timeout on `deepset-ai/haystack-8619`. The successful runs ranged from 23 to 171 worker steps. SWE-agent’s three passes were `conan-17092`, `haystack-8525`, and `conan-17129`. Several SWE-agent failures reached a submission but failed host-side tests; others encountered missing evaluation dependencies. The shared host-side verdict remains the reported outcome.

The result supports one bounded mechanism claim: executable judgment followed by evidence-driven repair can recover tasks that a cheaper flat loop leaves unresolved. It does not isolate the independent contribution of every subsystem. In particular, an early parallel-execution seam produced no gain in its controlled arm and was reverted. The paper therefore does not attribute the benchmark result to parallelism.

Matched ten-task SWE-bench-Live subset

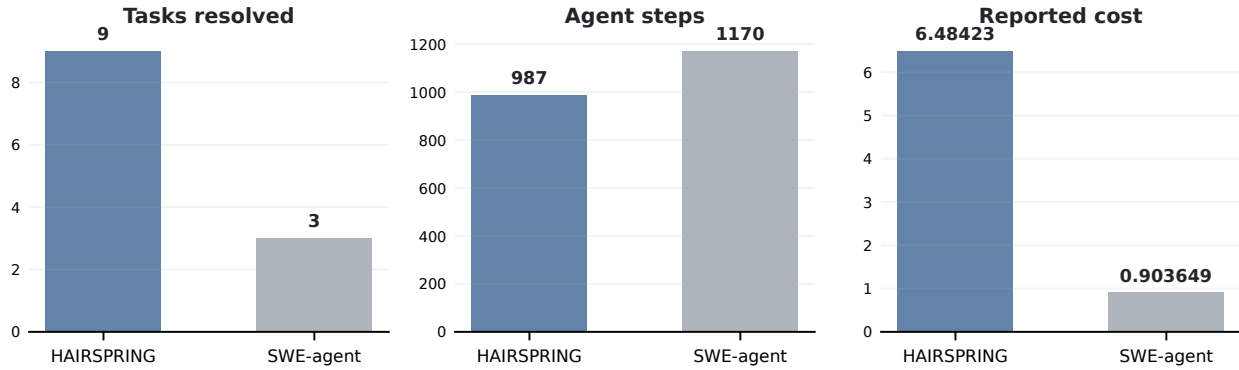


Figure 4: Matched ten-task results. HAIRSPRING resolved 9 tasks; SWE-agent resolved 3. HAIRSPRING used fewer agent steps and more provider-reported cost.

Table 2: Aggregate matched result. Wall time and cost are sums of the raw per-task records.

System	Resolved	Agent steps	Wall time	Cost (USD)
HAIRSPRING	9/10	987	26,935 s	6.484234
SWE-agent	3/10	1,170	3,613 s	0.903649

9.1 Per-task results

Table 3: HAIRSPRING per-task results from the checked-in raw CSV.

Instance	Pass	Steps	Calls	Wall (s)	Cost (millidollars)
cfn-lint-3764	yes	32	33	96	603
conan-17129	yes	61	62	158	769
haystack-8525	yes	23	24	67	397
haystack-8609	yes	60	61	237	1,302
conan-17092	yes	68	71	631	3,503
cfn-lint-3862	yes	86	89	349	3,024
streamlink-6439	yes	49	50	197	2,834
haystack-8799	yes	171	172	1,551	5,053
pdm-3314	yes	69	70	424	2,250
haystack-8619	no	368	368	2,774	7,200

10 Comparison with SWE-agent

SWE-agent is the reproducible incumbent for this study because it provides a public implementation, a defined agent-computer interface, and an executable run path. The systems solve different boundary problems. SWE-agent focuses the model’s interaction with a computer. HAIRSPRING adds in-run judgment, replay, shared-state authority, and outer-loop policy evaluation.

The trade is visible. HAIRSPRING spends more time and money to judge and repair during the run. SWE-agent remains simpler and cheaper when an external evaluator already supplies the authority and replay, memory, shared world state, and self-improvement are outside the job.

11 Failure-driven engineering

The highest-value defects crossed subsystem boundaries while local tests remained green. Four examples shaped the current test rule.

Table 4: Responsibility comparison. “Native” means the responsibility is a first-class harness path rather than a downstream benchmark function.

Responsibility	HAIRSPRING	SWE-agent
Computer interface	Typed process plugins; generated surface	Purpose-built agent–computer interface
Mission close	Executable checks plus fresh-context refutation	Agent submission; benchmark grades later
Recovery	Event replay restores counters, cost, transcript, and tools	Trajectory artifacts; resume is not the central abstraction
Shared state	Validated world proposals and provenance-bearing memory	No equivalent authority plane in the reference path
Parallel work	Bounded child streams with folded accounting	Single-agent reference path
Harness evolution	Held-out candidate/parent assays with pinned scorer	Outside the harness
Matched result	9/10 at \$6.48	3/10 at \$0.90

A dead tool surface. A capability could exist in a crate while the production terminal never registered it. The fix required a seam test through the real registry and TUI path.

A wrong production mode. General missions accidentally used an amnesiac benchmark arm. Unit tests for both modes passed; only a live installed-path run exposed the selection error.

Recursive delegation. The first real delegation test allowed descendants to delegate until the host reached roughly 7,800 processes. The repaired design removes delegation from a child’s surface and adds an explicit spawn cap.

An impossible gate. Missing test dependencies and strict marker settings caused the evaluator to collect zero tests while the agent kept repairing a correct patch. Preflight now proves that the gate can execute before a mission starts.

These failures lead to one rule: a mechanism is not accepted because its crate test passes. Its regression must reproduce how a model reaches it through the installed terminal path, including registry, sandbox, project root, judgment order, and rendered outcome.

12 Relationship to long-horizon research controllers

Stellar Colosseum organizes long-form research into strategy exploration, a readiness gate, a section dependency graph, local subproblem solving, and global verification. Its strongest transferable idea is explicit control over when a strategy is mature enough to decompose and where a verifier’s finding should return [2].

HAIRSPRING supplies a lower substrate for such a controller: replayable state, executable checks, a fresh critic, bounded children, shared world and memory planes, accounting, and held-out evolution. It does not yet implement Colosseum’s research-proof control plane. Missions remain flat. Critic findings return as whole-mission gaps. Candidates are selected or compared rather than constructively synthesized with their objections. No typed section graph localizes a defect to the affected dependency closure.

A research mode can be built above the current boundaries: strategy exploration → readiness decision → typed dependency graph → eligible child solves → global verification → affected-section repair or renewed exploration. The whole-artifact critic should remain final authority. Fixed tree widths should not become defaults without matched-compute evidence.

13 Limitations

1. The headline comparison contains ten curated tasks. It establishes a difference on that subset, not general software-engineering superiority.
2. Verification is expensive. On the matched subset, HAIRSPRING spent about 7.2 times as much as SWE-agent and took substantially longer.
3. The study has no repeated seeds and cannot estimate result variance.
4. Self-improvement changes prompts and policy artifacts, not the Rust implementation.

5. Memory writes, recall, and cross-child reuse are exercised end to end; live-model selection quality at scale is not measured.
6. The research controller is flat. Strategy readiness, typed section dependencies, objection-preserving synthesis, and section-addressed repair remain unimplemented.
7. At the evaluated paper revision, the workspace test suite passed but current pedantic Clippy produced 50 failures. Passing tests do not make a different quality gate pass.

14 Reproducibility

The repository carries manuscript source, generated figures, raw matched records, and a reproduction script. The source revision for the measurements is 4273dccb6d00fa727b01fcbdedc6773ace9eca12. Paper changes after that revision do not alter the raw run records.

Table 5: Regeneration commands. These commands do not call a paid model.

Measurement	Command from repository root
Rust source lines	<code>find crates -name '*.rs' -print0 xargs -0 wc -l</code>
Workspace crates	<code>find crates -mindepth 2 -maxdepth 2 -name Cargo.toml wc -l</code>
Named tests	<code>cargo test -workspace -locked - -list grep -c ': test\$'</code>
Workspace tests	<code>cargo test -workspace -locked -no-fail-fast</code>
Matched aggregates	<code>python3 paper/reproduce.py benchmark</code>
Paper PDF	<code>cd paper && make clean all</code>

The SWE-bench-Live runs require task repositories, test patches, provider credentials, and runner environments. The checked-in CSV and JSON permit recomputation of the aggregates without rerunning paid inference.

15 Discussion

Authority boundaries make failures legible. A failed check is not an answer-quality opinion; it is an execution fact. A critic malfunction is not a pass; it is a machinery state. A useful memory is not shared truth; it is a record with provenance and a reuse signal. A candidate with an attractive explanation is not an improvement; it is an assay input.

This framing does not remove model error. It changes where error can become state. The worker may still choose a poor plan or spend too long on a dead end. The substrate can preserve the evidence, enforce budgets, refuse an invalid close, and route a grounded defect back for repair. Those properties become more important as missions grow longer because recovery and attribution costs rise with the amount of hidden state.

16 Conclusion

HAIRSPRING treats agency as a problem of authority. The model does the open-ended work. A checker, critic, event record, world service, memory service, and pinned scorer decide what becomes a pass, durable state, shared knowledge, or an installed policy.

On one matched ten-task software-engineering subset, this architecture resolved more tasks than SWE-agent while using fewer worker steps, more money, and more wall time. The result is narrow. The design lesson is broader: a longer loop does not make its own verdict safer. Long-running agents become easier to inspect, recover, and improve when the loop proposes and the substrate decides.

A Artifact map

The implementation is divided by responsibility rather than by interface layer.

Crate	Responsibility
hs-core	Event schema, canonical encoding, hash chaining
hs-log	Segmented streams, payload store, fsync, recovery, rewind
hs-kernel	Plugin process isolation, protocol, dispatch
hs-loop	Worker loop, checker path, REPL, TUI, built-in tools and models
hs-swarm	Bounded asynchronous children, booking, join at close
hs-goal	Graded mission rig and external grading
hs-hashline, hs-applypatch	Anchored and patch-based editing
hs-bench, hs-scorer	Benchmarks, tiered scoring, pins, canaries, baselines
hs-selfmod	Candidate cycle, migration, promotion, rewind
hs-memory, hs-world	Provenance-bearing memory and validated shared state
hs-cli	Event-log inspection and fixtures

B Event and verdict flow

The following abbreviated trace shows the intended ordering. Field names are illustrative; the executable schema in `hs-core` is authoritative.

```
mission.opened    { goal, project_root, provider, budgets }
model.requested   { step, context_hash }
model.responded   { usage, billed_cost }
tool.requested    { name, args_hash }
tool.completed    { status, payload_hash }
answer.submitted  { artifact_hash, declared_checks }
checks.completed  { status, evidence_hash }
critic.completed  { refuted, findings_hash }
mission.closed    { outcome: "verified" | "rejected" | "stopped" }
```

A valid close follows persisted submission and judgment evidence. A replay reconstructs state from the ordered stream and content-addressed payloads. It does not ask the worker to summarize what happened.

C Matched raw-result schema

The HAIRSPRING CSV uses one row per task:

```
instance_id, passed, model_calls, steps, wall_ms, cost_microsd, note
```

The SWE-agent JSON stores the corresponding instance identifier, submission flag, host-side result, test tail, steps, cost, and token counts. `paper/reproduce.py benchmark` sums the two files and prints the totals reported in Table 2.

D Fresh-clone reproduction

```
git clone https://github.com/tensorbend-instinct/hairspring.git
cd hairspring
git checkout 4273dccf6d00fa727b01fcbdedc6773ace9eca12
export PATH="$HOME/.cargo/bin:$PATH"
```

```
cargo test --workspace --locked --no-fail-fast
python3 paper/reproduce.py source
python3 paper/reproduce.py benchmark
cd paper && make clean all
```

E Evaluation claims and boundaries

Claim	Status	Boundary
14 crates; 342 Rust files; 70,429 lines	Measured	Source tree at the evaluated revision
992 tests passed; 0 failed	Measured	Saved complete workspace run; named-test listing reports 994 entries
9/10 versus 3/10	Measured	One curated matched subset, same model family, host-side grading
Verifier–repair supports recovery	Supported	Mechanism-level interpretation; no component ablation isolates the full effect
Parallelism caused the gain	Not supported	First controlled seam was null and reverted
General software-engineering superiority	Not claimed	No random sample, repeated seeds, or confidence interval
Research-proof controller	Not implemented	Strategy readiness and section dependency state remain future work

References

- [1] J. Yang et al. “SWE-agent: Agent–Computer Interfaces Enable Automated Software Engineering.” *NeurIPS*, 2024. <https://arxiv.org/abs/2405.15793>.
- [2] H. Lin et al. “Stellar Colosseum: A Many-Agent Harness for Long-Horizon Research in Mathematics and Theoretical Computer Science.” 2026. <https://arxiv.org/abs/2609.15983>.
- [3] S. Pal et al. “SwarmWorld: Stigmergic technological evolution in societies of language-model agents.” 2026. <https://arxiv.org/abs/2608.26081>.
- [4] Y. Lee et al. “Meta-Harness: End-to-End Optimization of Model Harnesses.” 2026. <https://arxiv.org/html/2603.28052>.
- [5] Y. Ji. “Context Engineering for AI Agents: Lessons from Building Manus.” 2025. <https://manus.im/en/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>.
- [6] N. Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning.” *NeurIPS*, 2023.
- [7] S. Yao et al. “Tree of Thoughts: Deliberate Problem Solving with Large Language Models.” *NeurIPS*, 2023.
- [8] X. Wang et al. “Self-Consistency Improves Chain of Thought Reasoning in Language Models.” *ICLR*, 2023.
- [9] P. Gauthier. “Aider: AI pair programming in your terminal.” 2024. <https://aider.chat/>.

Citation

```
@software{hairspring2026,  
  title = {HAIRSPRING: Authority outside the model},  
  author = {{Tensorbend Instinct}},  
  year = {2026},  
  url = {https://github.com/tensorbend-instinct/hairspring},  
  note = {Evidence revision 4273dccf6d00fa727b01fcbdedc6773ace9eca12}  
}
```